

Syntax

```
options = trainingOptions(solverName)
options = trainingOptions(solverName,Name=Value)
```

Description

options = trainingOptions(solverName) returns training options for the optimizer specified by solverName. To train a neural network, use the training options as an input argument to the trainnet or trainNetwork function.

options = trainingOptions(solverName,Name=Value) returns training options with additional options specified by one or more name-value arguments.

Examples

Specify Training Options

Create a set of options for training a network using stochastic gradient descent with momentum. Reduce the learning rate by a factor of 0.2 every 5 epochs. Set the maximum number of epochs for training to 20, and use a mini-batch with 64 observations at each iteration. Turn on the training progress plot.

Try This Example

Copy Command

```
options = trainingOptions("sgdm", ...
    LearnRateSchedule="piecewise", ...
    LearnRateDropFactor=0.2, ...
    LearnRateDropPeriod=5, ...
    MaxEpochs=20, ...
    MiniBatchSize=64, ...
    Plots="training-progress")
```

Get

options =
TrainingOptionsSGDM with properties:

Momentum: 0.9000
InitialLearnRate: 0.0100
MaxEpochs: 20
LearnRateSchedule: 'piecewise'
LearnRateDropFactor: 0.2000
LearnRateDropPeriod: 5
MiniBatchSize: 64
Shuffle: 'once'
WorkerLoad: []
CheckpointFrequency: 1
CheckpointFrequencyUnit: 'epoch'
SequenceLength: 'longest'
DispatchInBackground: 0
L2Regularization: 1.0000e-04
GradientThresholdMethod: 'l2norm'
GradientThreshold: Inf
Verbose: 1
VerboseFrequency: 50
ValidationData: []
ValidationFrequency: 50
ValidationPatience: Inf
CheckpointPath: ''
ExecutionEnvironment: 'auto'
OutputFcn: []
Metrics: []
Plots: 'training-progress'
SequencePaddingValue: 0
SequencePaddingDirection: 'right'
InputDataFormats: "auto"
TargetDataFormats: "auto"
ResetInputNormalization: 1
BatchNormalizationStatistics: 'auto'
OutputNetwork: 'last-iteration'

Monitor Deep Learning Training Progress

This example shows how to monitor the training process of deep learning networks.

When you train networks for deep learning, it is often useful to monitor the training progress. By plotting various metrics during training, you can learn how the training is progressing. For example, you can determine if and how quickly the network accuracy is improving, and whether the network is starting to overfit the training data.

This example shows how to monitor training progress for networks trained using the trainNetwork function. For networks trained using a custom training loop, use a trainingProgressMonitor object to plot metrics during training. For more information, see Monitor Custom Training Loop Progress.

When you set the Plots training option to "training-progress" in trainingOptions and start network training, trainNetwork creates a figure and displays training metrics at every iteration. Each iteration is an estimation of the gradient and an update of the network parameters. If you specify validation data in trainingOptions, then the figure shows validation metrics each time trainNetwork validates the network. The figure plots the following:

- Training accuracy** — Classification accuracy on each individual mini-batch.
- Smoothed training accuracy** — Smoothed training accuracy, obtained by applying a smoothing algorithm to the training accuracy. It is less noisy than the unsmoothed accuracy, making it easier to spot trends.
- Validation accuracy** — Classification accuracy on the entire validation set (specified using trainingOptions).
- Training loss, smoothed training loss, and validation loss** — The loss on each mini-batch, its smoothed version, and the loss on the validation set, respectively. If the final layer of your network is a classificationLayer, then the loss function is the cross entropy loss. For more information about loss functions for classification and regression problems, see Output Layers.

For regression networks, the figure plots the root mean square error (RMSE) instead of the accuracy.

The figure marks each training **Epoch** using a shaded background. An epoch is a full pass through the entire data set.

During training, you can stop training and return the current state of the network by clicking the stop button in the top-right corner. For example, you might want to stop training when the accuracy of the network reaches a plateau and it is clear that the accuracy is no longer improving. After you click the stop button, it can take a while for the training to complete. Once training is complete, trainNetwork returns the trained network.

When training finishes, view the **Results** showing the finalized validation accuracy and the reason that training finished. If the OutputNetwork training option is "last-iteration" (default), the finalized metrics correspond to the last training iteration. If the OutputNetwork training option is "best-validation-loss", the finalized metrics correspond to the iteration with the lowest validation loss. The iteration from which the final validation metrics are calculated is labeled **Final** in the plots.

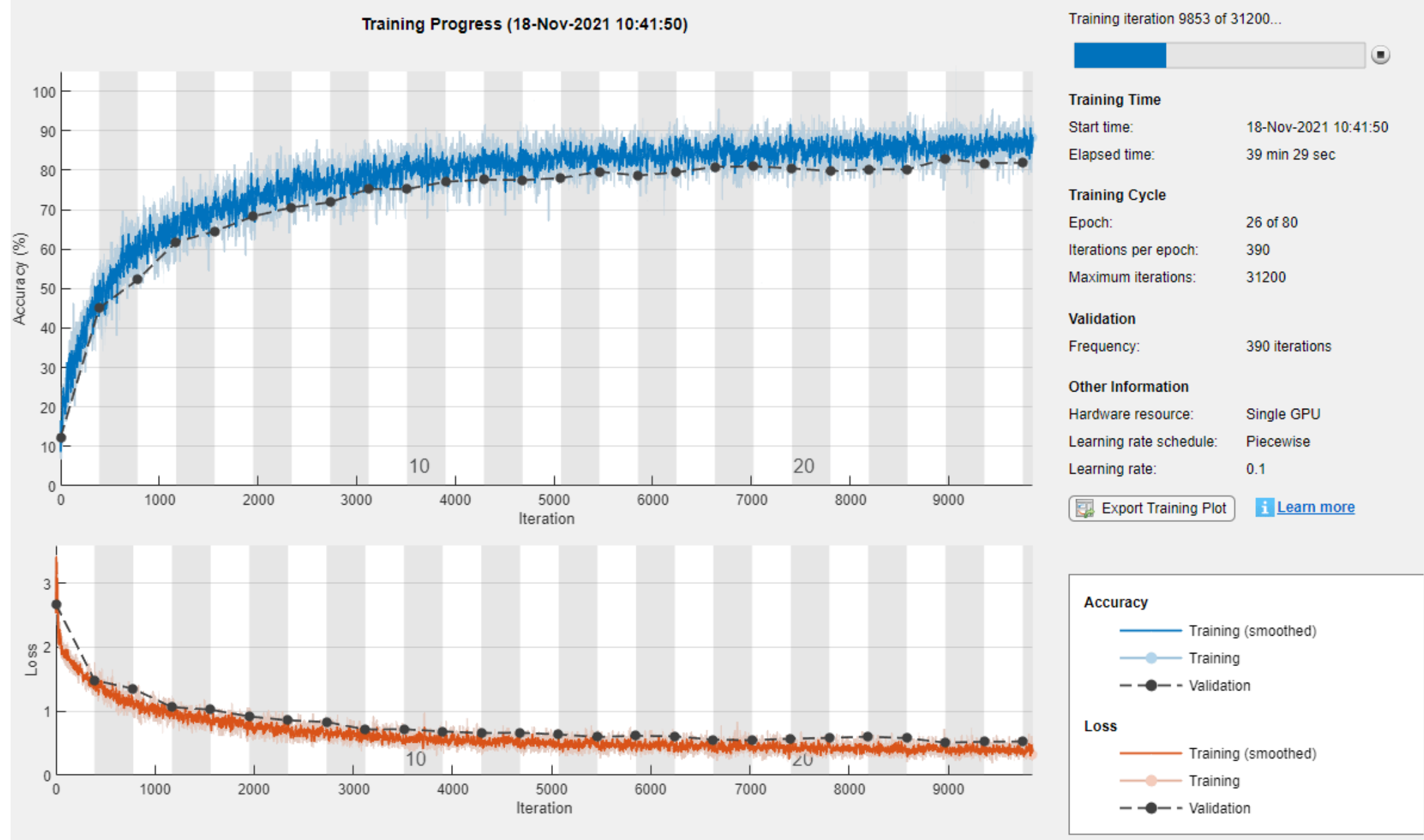
If your network contains batch normalization layers, then the final validation metrics can be different to the validation metrics evaluated during training. This is because the mean and variance statistics used for batch normalization can be different after training completes. For example, if the BatchNormalizationStatistics training option is "population", then after training, the software finalizes the batch normalization statistics by passing through the training data once more and uses the resulting mean and variance. If the BatchNormalizationStatistics training option is "moving", then the software approximates the statistics during training using a running estimate and uses the latest values of the statistics.

On the right, view information about the training time and settings. To learn more about training options, see Set Up Parameters and Train Convolutional Neural Network.

To save the training progress plot, click **Export Training Plot** in the training window. You can save the plot as a PNG, JPEG, TIFF, or PDF file. You can also save the individual plots of loss, accuracy, and root mean squared error using the axes toolbar.

Try This Example

Copy Command



Plot Training Progress During Training

Train a network and plot the training progress during training.

Load the training data, which contains 5000 images of digits. Set aside 1000 of the images for network validation.

```
[XTrain,YTrain] = digitTrain4DArrayData;

idx = randperm(size(XTrain,4),1000);
XValidation = XTrain(:,:,idx);
XTrain(:,:,idx) = [];
YValidation = YTrain(idx);
YTrain(idx) = [];
```

Construct a network to classify the digit image data.

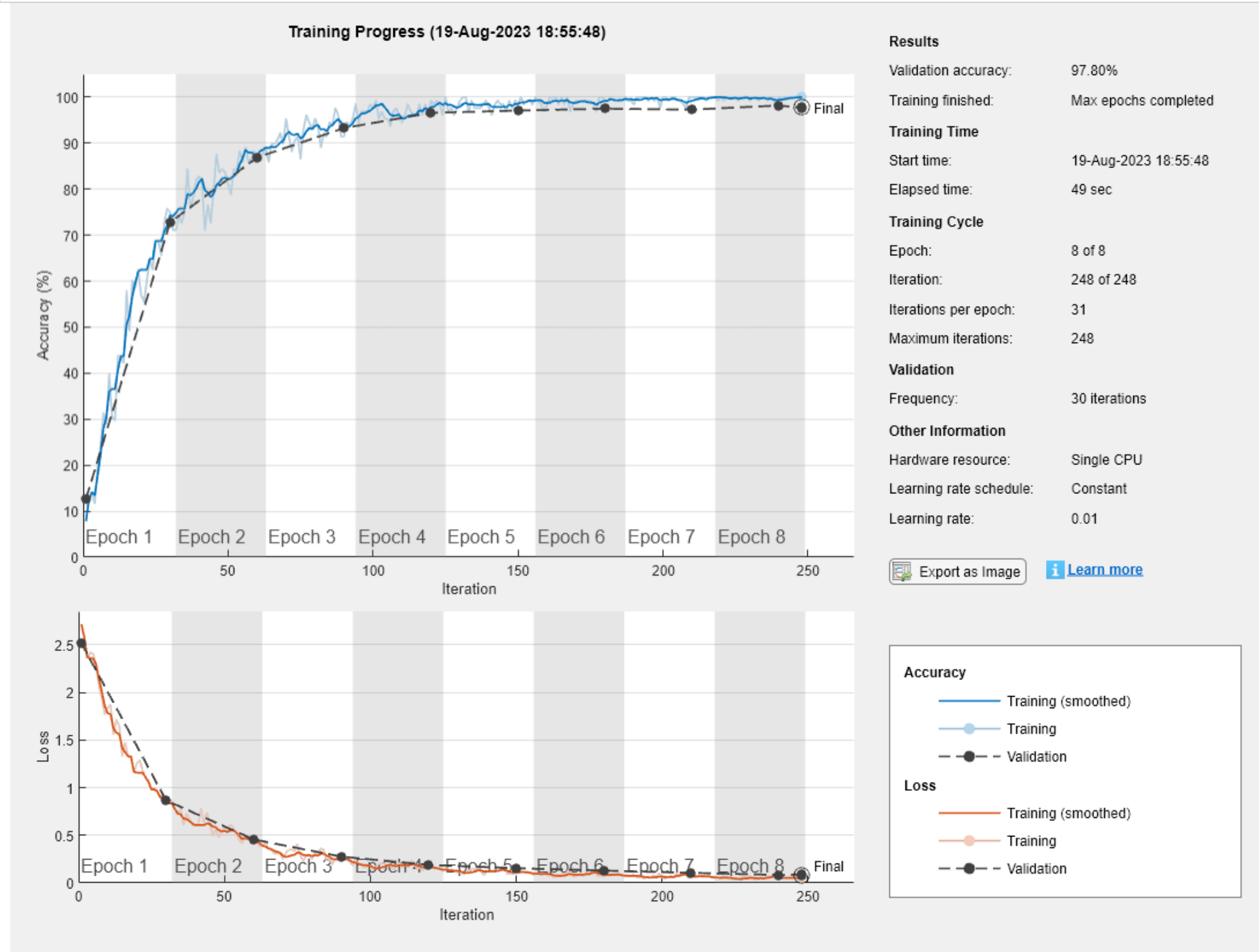
```
layers = [
    imageInputLayer([28 28 1])
    convolution2dLayer(3,8,Padding="same")
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2,Stride=2)
    convolution2dLayer(3,16,Padding="same")
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2,Stride=2)
    convolution2dLayer(3,32,Padding="same")
    batchNormalizationLayer
    reluLayer
    fullyConnectedLayer(10)
    softmaxLayer
    classificationLayer];
```

Specify options for network training. To validate the network at regular intervals during training, specify validation data. Choose the ValidationFrequency value so that the network is validated about once per epoch. To plot training progress during training, set the Plots training option to "training-progress".

```
options = trainingOptions("sgdm", ...
    MaxEpochs=8, ...
    ValidationData={XValidation,YValidation}, ...
    ValidationFrequency=30, ...
    Verbose=false, ...
    Plots="training-progress");
```

Train the network.

```
net = trainNetwork(XTrain,YTrain,layers,options);
```



collapse all

Input Arguments

▼ solverName — Solver for training neural network
"sgdm" | "rmsprop" | "adam" | "lbfgs"

Solver for training neural network, specified as one of these values:

- "sgdm" — Stochastic gradient descent with momentum (SGDM). SGDM is a stochastic solver. For additional training options, see Stochastic Solver Options. For more information, see Stochastic Gradient Descent with Momentum.
- "rmsprop" — Root mean square propagation (RMSProp). RMSProp is a stochastic solver. For additional training options, see Stochastic Solver Options. For more information, see Root Mean Square Propagation.
- "adam" — Adaptive moment estimation (Adam). Adam is a stochastic solver. For additional training options, see Stochastic Solver Options. For more information, see Adaptive Moment Estimation.
- "lbfgs" (*since R2023b*) — Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS). L-BFGS is a batch solver. The L-BFGS algorithm is best suited for small networks and data sets that you can process in a single batch. For additional training options, see L-BFGS Solver Options. For more information, see Limited-Memory BFGS. This option supports the `trainnet` function only.

The `trainBERTDocumentClassifier` (Text Analytics Toolbox) function supports the "sgdm", "rmsprop", and "adam" solvers only.

Name-Value Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose Name in quotes.

Example: `Plots="training-progress",Metrics="accuracy",Verbose=false` specifies to disable the verbose output and display the training progress in a plot that also includes the accuracy metric.

collapse all

Monitoring

▼ Plots — Plots to display during neural network training
"none" (default) | "training-progress"

Plots to display during neural network training, specified as one of these values:

- "none" — Do not display plots during training.
- "training-progress" — Plot training progress.

The contents of the plot depends on the training function that you use.

trainnet Function

- When the `solverName` argument is "sgdm", "adam", or "rmsprop", the plot shows the mini-batch loss, validation loss, training mini-batch and validation metrics specified by the `Metrics` option, and additional information about the training progress.
- When the `solverName` argument is "lbfgs", the plot shows the training and validation loss, training and validation metrics specified by the `Metrics` option, and additional information about the training progress.

To programmatically open and close the training progress plot after training, use the `show` and `close` functions with the second output of the `trainnet` function. You can use the `show` function to view the training progress even if the `Plots` training option is specified as "none".

trainNetwork Function

The plot shows the mini-batch loss and accuracy, validation loss and accuracy, and additional information about the training progress. For more information about the `trainNetwork` training progress plot, see Monitor Deep Learning Training Progress.

▼ Metrics — Metrics to track
[] (default) | character vector | string array | function handle | cell array | metric object

Since R2023b

Metrics to track, specified as a character vector or string scalar of a built-in metric name, a string array of names, a built-in or custom metric object, a function handle (`@myMetric`), or a cell array of names, metric objects, and function handles.

- Built-in metric name — Specify metrics as a string scalar, character vector, or string array of built-in metric names. Supported values are "accuracy", "fscore", "recall", "precision", "rmse", and "auc".
- Built-in metric object — If you need more flexibility, you can use built-in metric objects. The software supports these built-in metric objects:
 - `AccuracyMetric`
 - `AUCMetric`
 - `FScoreMetric`
 - `PrecisionMetric`
 - `RecallMetric`
 - `RMSEMetric`When you create a built-in metric object, you can specify additional options such as the averaging type and whether the task is single-label or multilabel.
- Custom metric function handle — If the metric you need is not a built-in metric, then you can specify custom metrics using a function handle. The function must have the syntax `metric = metricFunction(Y,T)`, where `Y` corresponds to the network predictions and `T` corresponds to the target responses. For networks with multiple outputs, the syntax must be `metric = metricFunction(Y1,...,YN,T1,...TM)`, where `N` is the number of outputs and `M` is the number of targets. For more information, see Define Custom Metric Function.

i

Note

When you have validation data in mini-batches, the software computes the validation metric for each mini-batch and then returns the average of those values. For some metrics, this behavior can result in a different metric value than if you compute the metric using the whole validation set at once. In most cases, the values are similar. To use a custom metric that is not batch-averaged for the validation data, you must create a custom metric object. For more information, see Define Custom Deep Learning Metric Object.

- Custom metric object — If you need greater customization, then you can define your own custom metric object. For an example that shows how to create a custom metric, see Define Custom F-Beta Score Metric Object . For general information about creating custom metrics, see Define Custom Deep Learning Metric Object. Specify your custom metric as the `Metrics` option of the `trainingOptions` function.

This option supports the `trainnet` and `trainBERTDocumentClassifier` (Text Analytics Toolbox) functions only.

Example: `Metrics=["accuracy","fscore"]`

Example: `Metrics=["accuracy",@myFunction,precisionObj]`

▼ Verbose — Flag to display training progress information
1 (true) (default) | 0 (false)

Flag to display training progress information in the command window, specified as `1` (true) or `0` (false).

The content of the verbose output depends on the function that you use for training.

trainnet Function

When you use the `trainnet` function, the verbose output displays a table. The variables of the table depends on the type of solver.

For stochastic solvers (SGDM, Adam, and RMSProp), the table contains these variables:

Variable	Description
Iteration	Iteration number
Epoch	Epoch number
TimeElapsed	Time elapsed in hours, minutes, and seconds
LearnRate	Learning rate
TrainingLoss	Training loss
ValidationLoss	Validation loss. If you do not specify validation data, then the software does not display this information.

For the L-BFGS solver, the table contains these variables:

Variable	Description
Iteration	Iteration number
TimeElapsed	Time elapsed in hours, minutes, and seconds
TrainingLoss	Training loss
ValidationLoss	Validation loss. If you do not specify validation data, then the software does not display this information.
GradientNorm	Norm of the gradients
StepNorm	Norm of the steps

If you specify additional metrics in the training options, then they also appear in the verbose output. For example, if you set the `Metrics` training option to "accuracy", then the information includes the `TrainingAccuracy` and `ValidationAccuracy` variables.

When training stops, the verbose output displays the reason for stopping.

To specify validation data, use the `ValidationData` training option.

trainNetwork Function

When you use the `trainNetwork` function, the verbose output displays a table. The variables of the table depends on the type of neural network.

For classification neural networks, the table contains these variables:

Variable	Description
Epoch	Epoch number. An epoch corresponds to a full pass of the data.
Iteration	Iteration number. An iteration corresponds to a mini-batch.
Time Elapsed	Time elapsed in hours, minutes, and seconds.
Mini-batch Accuracy	Classification accuracy on the mini-batch.
Validation Accuracy	Classification accuracy on the validation data. If you do not specify validation data, then the software does not display this information.
Mini-batch Loss	Loss on the mini-batch. If the output layer is a <code>ClassificationOutputLayer</code> object, then the loss is the cross entropy loss for multi-class classification problems with mutually exclusive classes.
Validation Loss	Loss on the validation data. If the output layer is a <code>ClassificationOutputLayer</code> object, then the loss is the cross entropy loss for multi-class classification problems with mutually exclusive classes. If you do not specify validation data, then the software does not display this information.
Base Learning Rate	Base learning rate. The software multiplies the learn rate factors of the layers by this value.

For regression neural networks, the table contains these variables:

Variable	Description
Epoch	Epoch number. An epoch corresponds to a full pass of the data.
Iteration	Iteration number. An iteration corresponds to a mini-batch.
Time Elapsed	Time elapsed in hours, minutes, and seconds.
Mini-batch RMSE	Root-mean-squared-error (RMSE) on the mini-batch.
Validation RMSE	RMSE on the validation data. If you do not specify validation data, then the software does not display this information.
Mini-batch Loss	Loss on the mini-batch. If the output layer is a <code>RegressionOutputLayer</code> object, then the loss is the half-mean-squared-error.
Validation Loss	Loss on the validation data. If the output layer is a <code>RegressionOutputLayer</code> object, then the loss is the half-mean-squared-error. If you do not specify validation data, then the software does not display this information.
Base Learning Rate	Base learning rate. The software multiplies the learn rate factors of the layers by this value.

When training stops, the verbose output displays the reason for stopping.

To specify validation data, use the `ValidationData` training option.

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64` | `logical`

▼ **VerboseFrequency — Frequency of verbose printing**
50 (default) | positive integer

Frequency of verbose printing, which is the number of iterations between printing to the command window, specified as a positive integer. This option only has an effect when the `Verbose` training option is `1` (`true`).

If you validate the neural network during training, then the software also prints to the command window every time validation occurs.

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

▼ **OutputFcn — Output functions**
function handle | cell array of function handles

Output functions to call during training, specified as a function handle or cell array of function handles. The software calls the functions once before the start of training, after each iteration, and once when training is complete.

The functions must have the syntax `stopFlag = f(info)`, where `info` is a structure containing information about the training progress, and `stopFlag` is a scalar that indicates to stop training early. If `stopFlag` is `1` (`true`), then the software stops training. Otherwise, the software continues training.

The fields of the structure `info` depend on the training function that you use.

trainnet Function

The `trainnet` function passes the output function the structure `info`.

For stochastic solvers (SGDM, Adam, and RMSProp), `info` contains these fields:

Field	Description
Epoch	Epoch number
Iteration	Iteration number
TimeElapsed	Time since start of training
LearnRate	Iteration learn rate
TrainingLoss	Iteration training loss
ValidationLoss	Validation loss, if specified and evaluated at iteration.
State	Iteration training state, specified as "start", "iteration", or "done".

For the L-BFGS solver, `info` contains these fields:

Field	Description
Iteration	Iteration number
TimeElapsed	Time elapsed in hours, minutes, and seconds
TrainingLoss	Training loss
ValidationLoss	Validation loss. If you do not specify validation data, then the software does not display this information.
GradientNorm	Norm of the gradients
StepNorm	Norm of the steps
State	Iteration training state, specified as "start", "iteration", or "done".

If you specify additional metrics in the training options, then they also appear in the training information. For example, if you set the `Metrics` training option to "accuracy", then the information includes the `TrainingAccuracy` and `ValidationAccuracy` fields.

If a field is not calculated or relevant for a certain call to the output functions, then that field contains an empty array.

For an example showing how to use output functions, see [Customize Output During Deep Learning Network Training](#).

trainNetwork Function

The `trainNetwork` function passes the output function the structure `info` that contains these fields:

Field	Description
Epoch	Current epoch number
Iteration	Current iteration number
TimeSinceStart	Time in seconds since the start of training
TrainingLoss	Current mini-batch loss
ValidationLoss	Loss on the validation data
BaseLearnRate	Current base learning rate
TrainingAccuracy	Accuracy on the current mini-batch (classification neural networks)
TrainingRMSE	RMSE on the current mini-batch (regression neural networks)
ValidationAccuracy	Accuracy on the validation data (classification neural networks)
ValidationRMSE	RMSE on the validation data (regression neural networks)
State	Current training state, with a possible value of "start", "iteration", or "done".

If a field is not calculated or relevant for the call to the output functions, then that field contains an empty array.

For an example showing how to use output functions, see [Customize Output During Deep Learning Network Training](#).

Data Types: `function_handle` | `cell`

Data Formats

[collapse all](#)

▼ **InputDataFormats — Description of input data dimensions**
"auto" (default) | string array | cell array of character vectors | character vector

Since R2023b

Description of the input data dimensions, specified as a string array, character vector, or cell array of character vectors.

If `InputDataFormats` is "auto", then the software uses the formats expected by the network input. Otherwise, the software uses the specified formats for the corresponding network input.

A data format is a string of characters, where each character describes the type of the corresponding dimension of the data.

The characters are:

S" — Spatial

C" — Channel

B" — Batch

T" — Time

U" — Unspecified

For example, for an array containing a batch of sequences where the first, second, and third dimension correspond to channels, observations, and time steps, respectively, you can specify that it has the format "CBT".

You can specify multiple dimensions labeled "S" or "U". You can use the labels "C", "B", and "T" at most once. The software ignores singleton trailing "U" dimensions located after the second dimension.

For more information, see Deep Learning Data Formats.

This option supports the `trainnet` function only.

Data Types: `char` | `string` | `cell`

TargetDataFormats — Description of target data dimensions

"auto" (default) | string array | cell array of character vectors | character vector

Since R2023b

Description of the target data dimensions, specified as one of these values:

"auto" — If the target data has the same number of dimensions as the input data, then the `trainnet` function uses the format specified by `InputDataFormats`. If the target data has a different number of dimensions to the input data, then the `trainnet` function uses the format expected by the loss function.

Data formats, specified as a string array, character vector, or cell array of character vectors — The `trainnet` function uses the specified data formats.

A data format is a string of characters, where each character describes the type of the corresponding dimension of the data.

The characters are:

S" — Spatial

C" — Channel

B" — Batch

T" — Time

U" — Unspecified

For example, for an array containing a batch of sequences where the first, second, and third dimension correspond to channels, observations, and time steps, respectively, you can specify that it has the format "CBT".

You can specify multiple dimensions labeled "S" or "U". You can use the labels "C", "B", and "T" at most once. The software ignores singleton trailing "U" dimensions located after the second dimension.

For more information, see Deep Learning Data Formats.

This option supports the `trainnet` function only.

Data Types: `char` | `string` | `cell`

Stochastic Solver Optionscollapse all

MaxEpochs — Maximum number of epochs

30 (default) | positive integer

Maximum number of epochs (full passes of the data) to use for training, specified as a positive integer.

This option supports stochastic solvers only (when the `solverName` argument is "sgdm", "adam", or "rmsprop").

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

MiniBatchSize — Size of mini-batch

128 (default) | positive integer

Size of the mini-batch to use for each training iteration, specified as a positive integer. A mini-batch is a subset of the training set that is used to evaluate the gradient of the loss function and update the weights.

If the mini-batch size does not evenly divide the number of training samples, then the software discards the training data that does not fit into the final complete mini-batch of each epoch. If the mini-batch size is smaller than the number of training samples, then the software does not discard any data.

This option supports stochastic solvers only (when the `solverName` argument is "sgdm", "adam", or "rmsprop").

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

Shuffle — Option for data shuffling

"once" (default) | "never" | "every-epoch"

Option for data shuffling, specified as one of these values:

"once" — Shuffle the training and validation data once before training.

"never" — Do not shuffle the data.

"every-epoch" — Shuffle the training data before each training epoch, and shuffle the validation data before each neural network validation. If the mini-batch size does not evenly divide the number of training samples, then the software discards the training data that does not fit into the final complete mini-batch of each epoch. To avoid discarding the same data every epoch, set the `Shuffle` training option to "every-epoch".

This option supports stochastic solvers only (when the `solverName` argument is "sgdm", "adam", or "rmsprop").

InitialLearnRate — Initial learning rate

positive scalar

Initial learning rate used for training, specified as a positive scalar.

If the learning rate is too low, then training can take a long time. If the learning rate is too high, then training might reach a suboptimal result or diverge.

This option supports stochastic solvers only (when the `solverName` argument is "sgdm", "adam", or "rmsprop").

When `solverName` is "sgdm", the default value is 0.01. When `solverName` is "rmsprop" or "adam", the default value is 0.001.

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

LearnRateSchedule — Option for dropping learning rate during training

"none" (default) | "piecewise"

Option for dropping the learning rate during training, specified as of these values:

"none" — Keep learning rate constant throughout training.

"piecewise" — Update the learning rate periodically by multiplying it by a drop factor. To specify the period, use the `LearnRateDropPeriod` training option. To specify the drop factor, use the `LearnRateDropFactor` training option.

This option supports stochastic solvers only (when the `solverName` argument is "sgdm", "adam", or "rmsprop").

LearnRateDropPeriod — Number of epochs for dropping the learning rate

10 (default) | positive integer

Number of epochs for dropping the learning rate, specified as a positive integer. This option is valid only when the `LearnRateSchedule` training option is "piecewise".

The software multiplies the global learning rate with the drop factor every time the specified number of epochs passes. Specify the drop factor using the `LearnRateDropFactor` training option.

This option supports stochastic solvers only (when the `solverName` argument is "sgdm", "adam", or "rmsprop").

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

LearnRateDropFactor — Factor for dropping the learning rate

0.1 (default) scalar from 0 to 1	
Factor for dropping the learning rate, specified as a scalar from 0 to 1. This option is valid only when the LearnRateSchedule training option is "piecewise".	
LearnRateDropFactor is a multiplicative factor to apply to the learning rate every time a certain number of epochs passes. Specify the number of epochs using the LearnRateDropPeriod training option.	
This option supports stochastic solvers only (when the solverName argument is "sgdm", "adam", or "rmsprop").	
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64	

✓ Momentum — Contribution of previous step 0.9 (default) scalar from 0 to 1
Contribution of the parameter update step of the previous iteration to the current iteration of stochastic gradient descent with momentum, specified as a scalar from 0 to 1.
A value of 0 means no contribution from the previous step, whereas a value of 1 means maximal contribution from the previous step. The default value works well for most tasks.
This option supports the SGDM solver only (when the solverName argument is "sgdm").
For more information, see Stochastic Gradient Descent with Momentum.
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64

✓ GradientDecayFactor — Decay rate of gradient moving average 0.9 (default) nonnegative scalar less than 1
Decay rate of gradient moving average for the Adam solver, specified as a nonnegative scalar less than 1. The gradient decay rate is denoted by β_1 in the Adaptive Moment Estimation section.
This option supports the Adam solver only (when the solverName argument is "adam").
For more information, see Adaptive Moment Estimation.
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64

✓ SquaredGradientDecayFactor — Decay rate of squared gradient moving average nonnegative scalar less than 1
Decay rate of squared gradient moving average for the Adam and RMSProp solvers, specified as a nonnegative scalar less than 1. The squared gradient decay rate is denoted by β_2 in [4].
Typical values of the decay rate are 0.9, 0.99, and 0.999, corresponding to averaging lengths of 10, 100, and 1000 parameter updates, respectively.
This option supports the Adam and RMSProp solvers only (when the solverName argument is "adam" or "rmsprop").
The default value is 0.999 for the Adam solver. The default value is 0.9 for the RMSProp solver.
For more information, see Adaptive Moment Estimation and Root Mean Square Propagation.
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64

✓ Epsilon — Denominator offset 1e-8 (default) positive scalar
Denominator offset for Adam and RMSProp solvers, specified as a positive scalar.
The solver adds the offset to the denominator in the neural network parameter updates to avoid division by zero. The default value works well for most tasks.
This option supports the Adam and RMSProp solvers only (when the solverName argument is "adam" or "rmsprop").
For more information, see Adaptive Moment Estimation and Root Mean Square Propagation.
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64

L-BFGS Solver Options

collapse all

✓ MaxIterations — Maximum number of iterations 1000 (default) positive integer
<i>Since R2023b</i>
Maximum number of iterations to use for training, specified as a positive integer.
The L-BFGS solver is a full-batch solver, which means that it processes the entire training set in a single iteration.
This option supports the L-BFGS solver only (when the solverName argument is "lbfgs").
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64

✓ LineSearchMethod — Method to find suitable learning rate "weak-wolfe" (default) "strong-wolfe" "backtracking"
<i>Since R2023b</i>
Method to find suitable learning rate, specified as one of these values:
• "weak-wolfe" — Search for a learning rate that satisfies the weak Wolfe conditions. This method maintains a positive definite approximation of the inverse Hessian matrix. • "strong-wolfe" — Search for a learning rate that satisfies the strong Wolfe conditions. This method maintains a positive definite approximation of the inverse Hessian matrix. • "backtracking" — Search for a learning rate that satisfies sufficient decrease conditions. This method does not maintain a positive definite approximation of the inverse Hessian matrix.
This option supports the L-BFGS solver only (when the solverName argument is "lbfgs").

✓ HistorySize — Number of state updates to store 10 (default) positive integer
<i>Since R2023b</i>
Number of state updates to store, specified as a positive integer. Values between 3 and 20 suit most tasks.
The L-BFGS algorithm uses a history of gradient calculations to approximate the Hessian matrix recursively. For more information, see Limited-Memory BFGS.
This option supports the L-BFGS solver only (when the solverName argument is "lbfgs").
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64

✓ InitialInverseHessianFactor — Initial value that characterizes approximate inverse Hessian matrix 1 (default) positive scalar
<i>Since R2023b</i>
Initial value that characterizes the approximate inverse Hessian matrix, specified as a positive scalar.
To save memory, the L-BFGS algorithm does not store and invert the dense Hessian matrix B . Instead, the algorithm uses the approximation $B_{k-m}^{-1} \approx \lambda_k I$, where m is the history size, the inverse Hessian factor λ_k is a scalar, and I is the identity matrix, and stores the scalar inverse Hessian factor only. The algorithm updates the inverse Hessian factor at each step.
The initial inverse hessian factor is the value of λ_0 .
For more information, see Limited-Memory BFGS.
This option supports the L-BFGS solver only (when the solverName argument is "lbfgs").
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64

MaxNumLineSearchIterations — Maximum number of line search iterations ✓
--

20 (default) positive integer	
Since R2023b	
Maximum number of line search iterations to determine learning rate, specified as a positive integer.	
This option supports the L-BFGS solver only (when the solverName argument is "lbfgs").	
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64	
✓	GradientTolerance — Relative gradient tolerance 1e-5 (default) positive scalar
Since R2023b	
Relative gradient tolerance, specified as a positive scalar.	
The software stops training when the relative gradient is less than or equal to GradientTolerance.	
This option supports the L-BFGS solver only (when the solverName argument is "lbfgs").	
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64	
✓	StepTolerance — Step size tolerance 1e-5 (default) positive scalar
Since R2023b	
Step size tolerance, specified as a positive scalar.	
The software stops training when the step taken is less than or equal to StepTolerance.	
This option supports the L-BFGS solver only (when the solverName argument is "lbfgs").	
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64	

Validation

collapse all

✓	ValidationData — Data to use for validation during training [] (default) datastore table cell array
Data to use for validation during training, specified as [], a datastore, a table, or a cell array containing the validation predictors and responses.	
During training, the software calculates the validation accuracy and validation loss on the validation data. To specify the validation frequency, use the ValidationFrequency training option. You can also use the validation data to stop training automatically when the validation loss stops decreasing. To turn on automatic validation stopping, use the ValidationPatience training option.	
If ValidationData is [], then the software does not validate the neural network during training.	
If your neural network has layers that behave differently during prediction than during training (for example, dropout layers), then the validation accuracy can be higher than the training accuracy.	
The validation data is shuffled according to the Shuffle training option. If Shuffle is "every-epoch", then the validation data is shuffled before each neural network validation.	
The supported formats depend on the training function that you use.	
trainnet Function Specify the validation data as a datastore or the cell array {predictors,targets}, where predictors contains the validation predictors and targets contains the validation targets. Specify the validation predictors and targets using any of the formats supported by the trainnet function. For more information, see the input arguments of the trainnet function.	
trainNetwork Function Specify the validation data as a datastore, table, or the cell array {predictors,targets}, where predictors contains the validation predictors and targets contains the validation targets. Specify the validation predictors and targets using any of the formats supported by the trainNetwork function. For more information, see the input arguments of the trainNetwork function.	
trainBERTDocumentClassifier Function (Text Analytics Toolbox) Specify the validation data as one of these values: - Cell array {documents,targets}, where documents contains the input documents, and targets contains the document labels - Table, where the first variable contains the input documents and the second variable contains the document labels. For more information, see the input arguments of the trainBERTDocumentClassifier (Text Analytics Toolbox) function.	

✓	ValidationFrequency — Frequency of neural network validation 50 (default) positive integer
Frequency of neural network validation in number of iterations, specified as a positive integer.	
The ValidationFrequency value is the number of iterations between evaluations of validation metrics. To specify validation data, use the ValidationData training option.	
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64	

✓	ValidationPatience — Patience of validation stopping Inf (default) positive integer
Patience of validation stopping of neural network training, specified as a positive integer or Inf.	
ValidationPatience specifies the number of times that the loss on the validation set can be larger than or equal to the previously smallest loss before neural network training stops. If ValidationPatience is Inf, then the values of the validation loss do not cause training to stop early.	
The returned neural network depends on the OutputNetwork training option. To return the neural network with the lowest validation loss, set the OutputNetwork training option to "best-validation-loss".	
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64	

✓	OutputNetwork — Neural network to return when training completes "last-iteration" (default) "best-validation-loss"
Neural network to return when training completes, specified as one of the following:	
"last-iteration" – Return the neural network corresponding to the last training iteration. "best-validation-loss" – Return the neural network corresponding to the training iteration with the lowest validation loss. To use this option, you must specify the ValidationData training option.	

Regularization and Normalization

collapse all

✓	L2Regularization — Factor for L₂ regularization 0.0001 (default) nonnegative scalar
Factor for L ₂ regularization (weight decay), specified as a nonnegative scalar. For more information, see L2 Regularization.	
You can specify a multiplier for the L ₂ regularization for neural network layers with learnable parameters. For more information, see Set Up Parameters in Convolutional and Fully Connected Layers.	
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64	

✓	ResetInputNormalization — Option to reset input layer normalization 1 (true) (default) 0 (false)
Option to reset input layer normalization, specified as one of the following:	
1 (true) – Reset the input layer normalization statistics and recalculate them at training time. 0 (false) – Calculate normalization statistics at training time when they are empty.	
Data Types: single double int8 int16 int32 int64 uint8 uint16 uint32 uint64 logical	

BatchNormalizationStatistics — Mode to evaluate statistics in batch normalization layers

"auto" (default) | "population" | "moving"

Mode to evaluate the statistics in batch normalization layers, specified as one of the following:

- "population" — Use the population statistics. After training, the software finalizes the statistics by passing through the training data once more and uses the resulting mean and variance.
- "moving" — Approximate the statistics during training using a running estimate given by update steps

$$\mu^* = \lambda_{\mu} \hat{\mu} + (1 - \lambda_{\mu}) \mu$$
$$\sigma^{2*} = \lambda_{\sigma^2} \widehat{\sigma^2} + (1 - \lambda_{\sigma^2}) \sigma^2$$

where μ^* and σ^{2*} denote the updated mean and variance, respectively, λ_{μ} and λ_{σ^2} denote the mean and variance decay values, respectively, $\hat{\mu}$ and $\widehat{\sigma^2}$ denote the mean and variance of the layer input, respectively, and μ and σ^2 denote the latest values of the moving mean and variance values, respectively. After training, the software uses the most recent value of the moving mean and variance statistics. This option supports CPU and single GPU training only.

- "auto" — Use the "moving" option for the `trainnet` function and the "population" option for the `trainNetwork` function.

Gradient Clipping

collapse all

GradientThreshold — Gradient threshold

Inf (default) | positive scalar

Gradient threshold, specified as Inf or a positive scalar. If the gradient exceeds the value of `GradientThreshold`, then the gradient is clipped according to the `GradientThresholdMethod` training option.

For more information, see [Gradient Clipping](#).

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

GradientThresholdMethod — Gradient threshold method

"l2norm" (default) | "global-l2norm" | "absolute-value"

Gradient threshold method used to clip gradient values that exceed the gradient threshold, specified as one of the following:

- "l2norm" — If the L₂ norm of the gradient of a learnable parameter is larger than `GradientThreshold`, then scale the gradient so that the L₂ norm equals `GradientThreshold`.
- "global-l2norm" — If the global L₂ norm, L , is larger than `GradientThreshold`, then scale all gradients by a factor of `GradientThreshold/L`. The global L₂ norm considers all learnable parameters.
- "absolute-value" — If the absolute value of an individual partial derivative in the gradient of a learnable parameter is larger than `GradientThreshold`, then scale the partial derivative to have magnitude equal to `GradientThreshold` and retain the sign of the partial derivative.

For more information, see [Gradient Clipping](#).

Sequence

collapse all

SequenceLength — Option to pad or truncate sequences

"longest" (default) | "shortest" | positive integer

Option to pad, truncate, or split input sequences, specified as one of these values:

- "longest" — Pad sequences in each mini-batch to have the same length as the longest sequence. This option does not discard any data, though padding can introduce noise to the neural network.
- "shortest" — Truncate sequences in each mini-batch to have the same length as the shortest sequence. This option ensures that no padding is added, at the cost of discarding data.
- Positive integer — For each mini-batch, pad the sequences to the length of the longest sequence in the mini-batch, and then split the sequences into smaller sequences of the specified length. If splitting occurs, then the software creates extra mini-batches. If the specified sequence length does not evenly divide the sequence lengths of the data, then the mini-batches containing the ends those sequences have length shorter than the specified sequence length. Use this option if the full sequences do not fit in memory. Alternatively, try reducing the number of sequences per mini-batch by setting the `MiniBatchSize` option to a lower value.
 - This option supports the `trainNetwork` function only.

To learn more about the effect of padding, truncating, and splitting the input sequences, see [Sequence Padding, Truncation, and Splitting](#).

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64` | `char` | `string`

SequencePaddingDirection — Direction of padding or truncation

"right" (default) | "left"

Direction of padding or truncation, specified as one of the following:

- "right" — Pad or truncate sequences on the right. The sequences start at the same time step and the software truncates or adds padding to the end of the sequences.
- "left" — Pad or truncate sequences on the left. The software truncates or adds padding to the start of the sequences so that the sequences end at the same time step.

Because recurrent layers process sequence data one time step at a time, when the recurrent layer `OutputMode` property is "last", any padding in the final time steps can negatively influence the layer output. To pad or truncate sequence data on the left, set the `SequencePaddingDirection` option to "left".

For sequence-to-sequence neural networks (when the `OutputMode` property is "sequence" for each recurrent layer), any padding in the first time steps can negatively influence the predictions for the earlier time steps. To pad or truncate sequence data on the right, set the `SequencePaddingDirection` option to "right".

To learn more about the effect of padding, truncating, and splitting the input sequences, see [Sequence Padding, Truncation, and Splitting](#).

SequencePaddingValue — Value to pad sequences

0 (default) | scalar

Value by which to pad input sequences, specified as a scalar.

Do not pad sequences with NaN, because doing so can propagate errors throughout the neural network.

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

Hardware

collapse all

ExecutionEnvironment — Hardware resource for training neural network

"auto" (default) | "cpu" | "gpu" | "multi-gpu" | "parallel" | "parallel-auto" | "parallel-cpu" | "parallel-gpu"

Hardware resource for training neural network, specified as one of these values:

Execution Environment	Hardware Resources Used
"auto"	Use a local GPU if one is available. Otherwise, use the local CPU.
"cpu"	Use the local CPU.
"gpu"	Use the local GPU.
"multi-gpu"	Use multiple GPUs on one machine, using a local parallel pool based on your default cluster profile. If there is no current parallel pool, the software starts a parallel pool with pool size equal to the number of available GPUs.
"parallel"	Use a local or remote parallel pool. If there is no current parallel pool, the software starts one using the default cluster profile. If the pool has access to GPUs, then only workers with a unique GPU perform training computation and excess workers become idle. If the pool does not have GPUs, then training takes place on all available CPU workers instead.
"parallel-auto"	- Use a local or remote parallel pool. If there is no current parallel pool, the software starts one using the default cluster profile. If the pool has access to GPUs, then only workers with a unique GPU perform training computation and excess workers become idle. If the pool does not have GPUs, then training takes place on all available CPU workers instead. - This option supports the <code>trainnet</code> function only.
"parallel-cpu"	- Use CPU resources in a local or remote parallel pool. If there is no current parallel pool, the software starts one using the default cluster profile. If the pool has access to GPUs, the GPUs will not be used. - This option supports the <code>trainnet</code> function only.
"parallel-gpu"	- Use GPUs in a local or remote parallel pool. Excess workers become idle. If there is no current parallel pool, the software starts one using the default cluster profile. - This option supports the <code>trainnet</code> function only.

The "gpu", "multi-gpu", "parallel", "parallel-auto", "parallel-cpu", and "parallel-gpu" options require Parallel Computing Toolbox™. To use a GPU for deep learning, you must also have a supported GPU device. For information on supported devices, see GPU Computing Requirements (Parallel Computing Toolbox). If you choose one of these options and Parallel Computing Toolbox or a suitable GPU is not available, then the software returns an error.

For more information on when to use the different execution environments, see [Scale Up Deep Learning in Parallel, on GPUs, and in the Cloud](#).

To see an improvement in performance when training in parallel, try scaling up the `MiniBatchSize` and `InitialLearnRate` training options by the number of GPUs.

When you train a network using the `trainNetwork` function, the `"multi-gpu"` and `"parallel"` options do not support neural networks containing custom layers with state parameters or built-in layers that are stateful at training time. For example:

- recurrent layers such as `LSTMLayer`, `BiLSTMLayer`, or `GRULayer` objects when the `SequenceLength` training option is a positive integer
- `BatchNormalizationLayer` objects when the `BatchNormalizationStatistics` training option is set to `"moving"`

The `"multi-gpu"`, `"parallel"`, `"parallel-auto"`, `"parallel-cpu"`, and `"parallel-gpu"` options support stochastic solvers only (when the `solverName` argument is `"sgdm"`, `"adam"`, or `"rmsprop"`).

WorkerLoad — Parallel worker load division

scalar from 0 to 1 | positive integer | numeric vector

Parallel worker load division between GPUs or CPUs, specified as one of the following:

- Scalar from 0 to 1 — Fraction of workers on each machine to use for neural network training computation. If you train the neural network using data in a mini-batch datastore with background dispatch enabled, then the remaining workers fetch and preprocess data in the background.
- Positive integer — Number of workers on each machine to use for neural network training computation. If you train the neural network using data in a mini-batch datastore with background dispatch enabled, then the remaining workers fetch and preprocess data in the background.
- Numeric vector — Neural network training load for each worker in the parallel pool. For a vector w , worker i gets a fraction $w(i)/\text{sum}(w)$ of the work (number of examples per mini-batch). If you train a neural network using data in a mini-batch datastore with background dispatch enabled, then you can assign a worker load of 0 to use that worker for fetching data in the background. The specified vector must contain one value per worker in the parallel pool.

If the parallel pool has access to GPUs, then workers without a unique GPU are never used for training computation. The default for pools with GPUs is to use all workers with a unique GPU for training computation, and the remaining workers for background dispatch. If the pool does not have access to GPUs and CPUs are used for training, then the default is to use one worker per machine for background data dispatch.

This option supports stochastic solvers only (when the `solverName` argument is `"sgdm"`, `"adam"`, or `"rmsprop"`).

This option supports the `trainNetwork` function only.

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

DispatchInBackground — Flag to enable background dispatch

0 (false) (default) | 1 (true)

Flag to enable background dispatch, specified as 0 (false) or 1 (true).

Background dispatch uses parallel workers to fetch and preprocess data from a datastore during training. Use this option when your mini-batches require significant preprocessing. For more information on when to use background dispatch, see [Use Datastore for Parallel Training and Background Dispatching](#).

When `DispatchInBackground` is set to `true`, the software opens a local parallel pool using the default profile, if a local pool is not currently open. Non-local parallel pools are not supported.

Using this option requires Parallel Computing Toolbox. The input datastore must be subsettable or partitionable. To use this option, custom datastores must implement the `matlab.io.datastore.Subsettable` class.

This option supports stochastic solvers only (when the `solverName` argument is `"sgdm"`, `"adam"`, or `"rmsprop"`).

This option does not support the `trainnet` function when training in parallel.

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

Checkpointscollapse all

CheckpointPath — Path for saving checkpoint neural networks

"" (default) | string scalar | character vector

Path for saving the checkpoint neural networks, specified as a string scalar or character vector.

- If you do not specify a path (that is, you use the default `""`), then the software does not save any checkpoint neural networks.
- If you specify a path, then the software saves checkpoint neural networks to this path and assigns a unique name to each neural network. You can then load any checkpoint neural network and resume training from that neural network.

If the folder does not exist, then you must first create it before specifying the path for saving the checkpoint neural networks. If the path you specify does not exist, then the software throws an error.

For more information about saving neural network checkpoints, see [Save Checkpoint Networks and Resume Training](#).

Data Types: `char` | `string`

CheckpointFrequency — Frequency of saving checkpoint neural networks

positive integer

Frequency of saving checkpoint neural networks, specified as a positive integer.

If `solverName` is `"lbfgs"` or `CheckpointFrequencyUnit` is `"iteration"`, then the software saves checkpoint neural networks every `CheckpointFrequency` iterations. Otherwise, the software saves checkpoint neural networks every `CheckpointFrequency` epochs.

When `solverName` is `"sgdm"`, `"adam"`, or `"rmsprop"`, the default value is 1. When `solverName` is `"lbfgs"`, default value is 30.

This option only has an effect when `CheckpointPath` is nonempty.

Data Types: `single` | `double` | `int8` | `int16` | `int32` | `int64` | `uint8` | `uint16` | `uint32` | `uint64`

CheckpointFrequencyUnit — Checkpoint frequency unit

"epoch" (default) | "iteration"

Checkpoint frequency unit, specified as `"epoch"` or `"iteration"`.

If `CheckpointFrequencyUnit` is `"epoch"`, then the software saves checkpoint neural networks every `CheckpointFrequency` epochs.

If `CheckpointFrequencyUnit` is `"iteration"`, then the software saves checkpoint neural networks every `CheckpointFrequency` iterations.

This option only has an effect when `CheckpointPath` is nonempty.

This option supports stochastic solvers only (when the `solverName` argument is `"sgdm"`, `"adam"`, or `"rmsprop"`).

Output Argumentscollapse all

options — Training options

TrainingOptionsSGDM | TrainingOptionsRMSProp | TrainingOptionsADAM | TrainingOptionsLBFGS

Training options, returned as a `TrainingOptionsSGDM`, `TrainingOptionsRMSProp`, `TrainingOptionsADAM`, or `TrainingOptionsLBFGS` object. To train a neural network, use the training options as an input argument to the `trainNetwork` or `trainnet` function. `TrainingOptionsLBFGS` objects support the `trainnet` function only.

If `solverName` is `"sgdm"`, `"rmsprop"`, `"adam"`, or `"lbfgs"`, then the training options are returned as a `TrainingOptionsSGDM`, `TrainingOptionsRMSProp`, `TrainingOptionsADAM`, or `TrainingOptionsLBFGS` object, respectively.

Tips

- For most deep learning tasks, you can use a pretrained neural network and adapt it to your own data. For an example showing how to use transfer learning to retrain a convolutional neural network to classify a new set of images, see [Train Deep Learning Network to Classify New Images](#). Alternatively, you can create and train neural networks from scratch using the `trainnet`, `trainNetwork`, and `trainingOptions` functions.
- If the `trainingOptions` function does not provide the training options that you need for your task, then you can create a custom training loop using automatic differentiation. To learn more, see [Define Deep Learning Network for Custom Training Loops](#).

Algorithmscollapse all

Initial Weights and Biases

For convolutional and fully connected layers, the initialization for the weights and biases are given by the `WeightsInitializer` and `BiasInitializer` properties of the layers, respectively. For examples showing how to change the initialization for the weights and biases, see [Specify Initial Weights and Biases in Convolutional Layer](#) and [Specify Initial Weights and Biases in Fully Connected Layer](#).

Stochastic Gradient Descent

The standard gradient descent algorithm updates the network parameters (weights and biases) to minimize the loss function by taking small steps at each iteration in the direction of the negative gradient of the loss,
$$\theta_{\ell+1} = \theta_{\ell} - \alpha \nabla E(\theta_{\ell}),$$
where ℓ is the iteration number, $\alpha > 0$ is the learning rate, θ is the parameter vector, and $E(\theta)$ is the loss function. In the standard gradient descent algorithm, the gradient of the loss function, $\nabla E(\theta)$, is evaluated using the entire training set, and the standard gradient descent algorithm uses the entire data set at once.

By contrast, at each iteration the *stochastic* gradient descent algorithm evaluates the gradient and updates the parameters using a subset of the training data. A different subset, called a mini-batch, is used at each iteration. The full pass of the training algorithm over the entire training set using mini-batches is one *epoch*. Stochastic gradient descent is stochastic because the parameter updates computed using a mini-batch is a noisy estimate of the parameter update that would result from using the full data set.

Stochastic Gradient Descent with Momentum

The stochastic gradient descent algorithm can oscillate along the path of steepest descent towards the optimum. Adding a momentum term to the parameter update is one way to reduce this oscillation [2]. The stochastic gradient descent with momentum (SGDM) update is
$$\theta_{\ell+1} = \theta_{\ell} - \alpha \nabla E(\theta_{\ell}) + \gamma(\theta_{\ell} - \theta_{\ell-1}),$$

where the learning rate α and the momentum value γ determine the contribution of the previous gradient step to the current iteration.

√ **Root Mean Square Propagation**

Stochastic gradient descent with momentum uses a single learning rate for all the parameters. Other optimization algorithms seek to improve network training by using learning rates that differ by parameter and can automatically adapt to the loss function being optimized. Root mean square propagation (RMSProp) is one such algorithm. It keeps a moving average of the element-wise squares of the parameter gradients,

$$v_{\ell} = \beta_2 v_{\ell-1} + (1 - \beta_2)[\nabla E(\theta_{\ell})]^2$$

β_2 is the squared gradient decay factor of the moving average. Common values of the decay rate are 0.9, 0.99, and 0.999. The corresponding averaging lengths of the squared gradients equal $1/(1-\beta_2)$, that is, 10, 100, and 1000 parameter updates, respectively. The RMSProp algorithm uses this moving average to normalize the updates of each parameter individually,

$$\theta_{\ell+1} = \theta_{\ell} - \frac{\alpha \nabla E(\theta_{\ell})}{\sqrt{v_{\ell} + \epsilon}}$$

where the division is performed element-wise. Using RMSProp effectively decreases the learning rates of parameters with large gradients and increases the learning rates of parameters with small gradients. ϵ is a small constant added to avoid division by zero.

√ **Adaptive Moment Estimation**

Adaptive moment estimation (Adam) [4] uses a parameter update that is similar to RMSProp, but with an added momentum term. It keeps an element-wise moving average of both the parameter gradients and their squared values,

$$m_{\ell} = \beta_1 m_{\ell-1} + (1 - \beta_1) \nabla E(\theta_{\ell})$$

$$v_{\ell} = \beta_2 v_{\ell-1} + (1 - \beta_2)[\nabla E(\theta_{\ell})]^2$$

The β_1 and β_2 decay rates are the gradient decay and squared gradient decay factors, respectively. Adam uses the moving averages to update the network parameters as

$$\theta_{\ell+1} = \theta_{\ell} - \frac{\alpha m_{\ell}}{\sqrt{v_{\ell} + \epsilon}}$$

The value α is the learning rate. If gradients over many iterations are similar, then using a moving average of the gradient enables the parameter updates to pick up momentum in a certain direction. If the gradients contain mostly noise, then the moving average of the gradient becomes smaller, and so the parameter updates become smaller too. The full Adam update also includes a mechanism to correct a bias the appears in the beginning of training. For more information, see [4].

√ **Limited-Memory BFGS**

The L-BFGS algorithm [5] is a quasi-Newton method that approximates the Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm. The L-BFGS algorithm is best suited for small networks and data sets that you can process in a single batch.

The algorithm updates learnable parameters W at iteration $k+1$ using the update step given by

$$W_{k+1} = W_k - \eta_k B_k^{-1} \nabla J(W_k),$$

where W_k denotes the weights at iteration k , η_k is the learning rate at iteration k , B_k is an approximation of the Hessian matrix at iteration k , and $\nabla J(W_k)$ denotes the gradients of the loss with respect to the learnable parameters at iteration k .

The L-BFGS algorithm computes the matrix-vector product $B_k^{-1} \nabla J(W_k)$ directly. The algorithm does not require computing the inverse of B_k .

To save memory, the L-BFGS algorithm does not store and invert the dense Hessian matrix B . Instead, the algorithm uses the approximation $B_{k-m}^{-1} \approx \lambda_k I$, where m is the history size, the inverse Hessian factor λ_k is a scalar, and I is the identity matrix, and stores the scalar inverse Hessian factor only. The algorithm updates the inverse Hessian factor at each step.

To compute the matrix-vector product $B_k^{-1} \nabla J(W_k)$ directly, the L-BFGS algorithm uses this recursive algorithm:

1. Set $\mathbf{r} = B_{k-m}^{-1} \nabla J(W_k)$, where m is the history size.
2. For $i = m, \dots, 1$:
 - a. Let $\beta = \frac{1}{\mathbf{s}_{k-i}^T \mathbf{y}_{k-i}} \mathbf{y}_{k-i}^T \mathbf{r}$, where $\mathbf{s}_{k-i}$ and $\mathbf{y}_{k-i}$ are the step and gradient differences for iteration $k-i$, respectively.
 - b. Set $\mathbf{r} = \mathbf{r} + \mathbf{s}_{k-i}(\mathbf{a}_{k-i} - \beta)$, where $\mathbf{a}$ is derived from $\mathbf{s}$, $\mathbf{y}$, and the gradients of the loss with respect to the loss function. For more information, see [5].
3. Return $B_k^{-1} \nabla J(W_k) = \mathbf{r}$.

√ **Gradient Clipping**

If the gradients increase in magnitude exponentially, then the training is unstable and can diverge within a few iterations. This "gradient explosion" is indicated by a training loss that goes to NaN or Inf. Gradient clipping helps prevent gradient explosion by stabilizing the training at higher learning rates and in the presence of outliers [3]. Gradient clipping enables networks to be trained faster, and does not usually impact the accuracy of the learned task.

There are two types of gradient clipping.

- Norm-based gradient clipping rescales the gradient based on a threshold, and does not change the direction of the gradient. The "l2norm" and "global-l2norm" values of GradientThresholdMethod are norm-based gradient clipping methods.
- Value-based gradient clipping clips any partial derivative greater than the threshold, which can result in the gradient arbitrarily changing direction. Value-based gradient clipping can have unpredictable behavior, but sufficiently small changes do not cause the network to diverge. The "absolute-value" value of GradientThresholdMethod is a value-based gradient clipping method.

√ **L₂ Regularization**

Adding a regularization term for the weights to the loss function $E(\theta)$ is one way to reduce overfitting [1], [2]. The regularization term is also called *weight decay*. The loss function with the regularization term takes the form

$$E_R(\theta) = E(\theta) + \lambda \Omega(w),$$

where w is the weight vector, λ is the regularization factor (coefficient), and the regularization function $\Omega(w)$ is

$$\Omega(w) = \frac{1}{2} w^T w.$$

Note that the biases are not regularized [2]. You can specify the regularization factor λ by using the L2Regularization training option. You can also specify different regularization factors for different layers and parameters. For more information, see Set Up Parameters in Convolutional and Fully Connected Layers.

The loss function that the software uses for network training includes the regularization term. However, the loss value displayed in the command window and training progress plot during training is the loss on the data only and does not include the regularization term.

References

[1] Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer, New York, NY, 2006.

[2] Murphy, K. P. *Machine Learning: A Probabilistic Perspective*. The MIT Press, Cambridge, Massachusetts, 2012.

[3] Pascanu, R., T. Mikolov, and Y. Bengio. "On the difficulty of training recurrent neural networks". *Proceedings of the 30th International Conference on Machine Learning*. Vol. 28(3), 2013, pp. 1310–1318.

[4] Kingma, Diederik, and Jimmy Ba. "Adam: A method for stochastic optimization." *arXiv preprint arXiv:1412.6980* (2014).

[5] Liu, Dong C., and Jorge Nocedal. "On the limited memory BFGS method for large scale optimization." *Mathematical programming* 45, no. 1 (August 1989): 503-528. <https://doi.org/10.1007/BF01589116>.

Version History

Introduced in R2016a	expand all
> R2023b: Train neural network using L-BFGS solver	
> R2023b: Specify input and target data formats	
> R2023b: Train neural network in parallel using only CPU or only GPU resources	
> R2023b: BatchNormalizationStatistics default is "auto" 	
> R2022b: trainNetwork pads mini-batches to length of longest sequence before splitting when you specify SequenceLength training option as an integer 	
> R2018b: ValidationPatience training option default is Inf 	
> R2018b: Different file name for checkpoint networks 	

See Also

trainnet | trainNetwork | analyzeNetwork | Deep Network Designer

Topics

- Create Simple Deep Learning Neural Network for Classification
- Transfer Learning Using Pretrained Network
- Resume Training from Checkpoint Network
- Deep Learning with Big Data on CPUs, GPUs, in Parallel, and on the Cloud
- Specify Layers of Convolutional Neural Network
- Set Up Parameters and Train Convolutional Neural Network
- Define Custom Training Loops, Loss Functions, and Networks

